



Database Views & Normalization

Overview

In this assignment, you will enhance your group's database application by implementing SQL views for controlled data access and applying normalization techniques to achieve Third Normal Form (3NF). You will create views to restrict access to specific data and write SQL commands for managing privileges. Additionally, you will analyze your database for normalization violations, make necessary modifications to improve data integrity, and document your work with SQL queries, screenshots, and a detailed explanation of your normalization process.

Part I: SQL Views & Authorization

In this part of the assignment, you will create views in MySQL to control data access and enhance security. Views allow you to present a subset of your data to specific users without granting them full access to the underlying tables. This is particularly useful for protecting sensitive information while still providing necessary data access to different types of users.

Since the MySQL version in your cPanel does not support role-based privileges or granting user-specific permissions through phpMyAdmin, you will also write (but not execute) SQL commands that would typically be used to manage access.

A. Setup

Before creating views, complete the following:

- ☐ Complete the [Logging Into cPanel section](#) to access your cPanel account.
- ☐ Since you will be creating views, you need to verify that your database user has the necessary privileges. Complete the [Updating A User's Privileges section](#) and ensure your database user has been granted **All Privileges** in phpMyAdmin.
- ☐ Complete the [Accessing The Database section](#) to access the database you created.
- ☐ Review the [SQL Statements section](#) to familiarize with common SQL statements to effectively communicate with your database.
- ☐ Review the [Querying The Database section](#) to write queries and ask questions about the data.

B. Creating Views

You will create at least two views that restrict access to certain data. Consider the following:

- A **limited-access view** that hides sensitive information (i.e., removing customer payment details).
- A **role-based view** that only displays records relevant to a specific user type (i.e., showing only active rental transactions for employees).

After creating your views, they will appear under the **Views** section at the bottom of your database.

C. Simulating Role-Based Access (Written SQL Only, No Execution)

Since the MySQL version in cPanel does not support granting privileges or creating roles, you will write (but not execute) the SQL commands that would typically be used to:

- Grant privileges on a view to a specific user.

```
-- Grant access to the email view only to research group members

GRANT      SELECT      ON      research_group_db.project_access_view      TO
'user@localhost';

-- Apply the changes

FLUSH PRIVILEGES;
```

- Create a role and assign users to it.

```
-- Create roles

CREATE ROLE 'project_lead';

CREATE ROLE 'researcher';

-- Grant view access to each role

-- Project leads can see both views

GRANT      SELECT      ON      research_group_db.member_contact_view      TO
'project_lead';

GRANT      SELECT      ON      research_group_db.project_access_view      TO
'project_lead';

-- Researchers can only see the project access view (no emails)
```

```
GRANT      SELECT      ON      research_group_db.project_access_view      TO  
'researcher';
```

- Assign roles to users

```
GRANT 'project_lead' TO 'Prof@localhost';
```

```
GRANT 'researcher' TO 'user@localhost';
```

- Apply changes

```
FLUSH PRIVILEGES;
```

Even though these commands will not execute in your current setup, understanding them is essential for real-world database management.

Part II: Database Normalization

In this part of the assignment, you will analyze and refine your group's database application design to ensure compliance with third normal form (3NF) by identifying and addressing normalization issues.

A. Database Normalization

Normalization helps eliminate redundancy, inconsistencies, and update anomalies by organizing data efficiently. You will review your current database design and apply normalization techniques as needed.

Familiarize yourself with the following rules to analyze your database for normalization violations:

- **First Normal Form (1NF)**
 - Using row order to convey information is not permitted.
 - Mixing data types within the same column is not permitted.
 - Having a table without a primary key is not permitted.
 - Storing repeating groups or multi-valued attributes is not permitted.
- **Second Normal Form (2NF)**
 - Each non-key attribute must depend on the entire primary key.
- **Third Normal Form (3NF)**
 - Every non-key attribute in a table should depend on the key, the whole key and nothing but the key.

Identify non-conforming elements in your existing database schema. Modify tables by restructuring attributes and relationships as needed to ensure that your final design meets 3NF requirements while maintaining functionality. This may involve breaking down tables, creating new tables, and redefining relationships to eliminate redundancy while ensuring data integrity.

As of now 1NF and 2NF are applied and in use for our database,. 3NF is also applied as well. Even though all normal forms are applied in order to improve database structure we can split the experiment table in 2 to make it easier to access.

Part III: Documentation

You will compile and explain your SQL queries, screenshots, written privilege management commands, and normalization analysis.

1. Document the exact SQL queries you used to define each view. Include a brief explanation of each view's purpose and expected results.

```
1089 + -- -----
1090 + -- View 1: member_contact_view
1091 + -- Only research group members can see emails
1092 + -- -----
1093 + CREATE VIEW `member_contact_view` AS
1094 + SELECT
1095 +     p.person_id,
1096 +     p.name,
1097 +     p.email,
1098 +     pg.role,
1099 +     rg.name AS group_name
1100 + FROM person p
1101 + JOIN person_group pg ON p.person_id = pg.person_id
1102 + JOIN research_group rg ON pg.group_id = rg.group_id;
```

This view displays the contact information for people who are active members of a research group; its purpose is to allow group members to see their colleagues' contact details and roles without exposing the full person table. Anyone without a group member won't appear here at all. The expected results: one row per person/group membership showing the person's ID, name, email, role, and the name of their research group.

```
1104 + -- -----
1105 + -- View 2: project_access_view
1106 + -- Project Leads see all projects,
1107 + -- everyone else only sees their assigned projects
1108 + -- -----
1109 + CREATE VIEW `project_access_view` AS
1110 + SELECT
1111 +     p.person_id,
1112 +     p.name,
1113 +     pm.role AS project_role,
1114 +     proj.project_id,
1115 +     proj.title,
1116 +     proj.description,
1117 +     proj.status,
1118 +     proj.start_date,
1119 +     proj.end_date
1120 + FROM person p
1121 + JOIN project_member pm ON p.person_id = pm.person_id
1122 + JOIN project proj ON (
1123 +     -- Project Leads see all projects
1124 +     pm.role = 'Project Lead'
1125 +     OR
1126 +     -- Everyone else only sees their assigned projects
1127 +     proj.project_id = pm.project_id
1128 + );
1129 +
```

This view enforces role based access to project data; it ensures that project leads can see all the projects in the database, while everyone else only sees the projects they are directly assigned to. Expected results: one row per person/project assignment displaying the person's ID, name, project role, project ID, title, description, status, start date, and end date.

2. Take screenshots showing the data displayed under each view. You can use the **Browse** tab and take a screenshot of the data that is displayed under the **Views** section. This will demonstrate that your views were successfully created.

Showing rows 0 - ... (Query took 0.0005 seconds.)

```
SELECT * FROM `member_contact_view`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

> | ☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	person_id	name	email	role	group_name
☐ Edit Copy Delete	1	Alice Johnson	alice.johnson@uni.edu	Principal Investigator	Genomics Lab
☐ Edit Copy Delete	2	Bob Martinez	bob.martinez@uni.edu	Principal Investigator	Quantum Dynamics Group
☐ Edit Copy Delete	3	Carol Chen	carol.chen@uni.edu	Lead Data Scientist	AI in Medicine
☐ Edit Copy Delete	4	David Kim	david.kim@uni.edu	Senior Researcher	Climate Modeling Team
☐ Edit Copy Delete	5	Eva Patel	eva.patel@uni.edu	Principal Investigator	Neural Circuits Lab
☐ Edit Copy Delete	6	Frank Müller	frank.muller@uni.edu	Researcher	Biomaterials Research Unit
☐ Edit Copy Delete	7	Grace Okafor	grace.okafor@uni.edu	Research Associate	Data Mining Group
☐ Edit Copy Delete	8	Henry Zhao	henry.zhao@uni.edu	Senior Researcher	Photonics Lab
☐ Edit Copy Delete	9	Isla Rivera	isla.rivera@uni.edu	Postdoctoral Fellow	Structural Biology Group
☐ Edit Copy Delete	10	James Novak	james.novak@uni.edu	Principal Investigator	Computational Chemistry Team

Showing rows 0 - ... (Query took 0.0005 seconds.)

```
SELECT * FROM `project_access_view`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

> | ☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	person_id	name	project_role	project_id	title	description	status	start_date	end_date
☐ Edit Copy Delete	1	Alice Johnson	Project Lead	1	CRISPR Gene Editing Study	Investigating CRISPR-Cas9 efficacy in human cell l...	Active	2023-01-15	2024-12-31
☐ Edit Copy Delete	1	Alice Johnson	Project Lead	2	Quantum Error Correction	Developing algorithms to minimize quantum decohere...	Active	2022-06-01	2025-05-31
☐ Edit Copy Delete	1	Alice Johnson	Project Lead	3	AI Tumor Detection	CNN-based model for early-stage tumor detection in...	Completed	2023-03-01	2024-09-30
☐ Edit Copy Delete	1	Alice Johnson	Project Lead	4	Arctic Ice Melt Simulation	Modeling ice sheet dynamics under	Completed	2021-09-01	2023-08-31

3. Include the SQL statements you would have executed to grant privileges on a view and create a role. Document each query's intended purpose, expected results, and actual SQL command.

- The purpose of granting a specific user the permission to access a certain view, such as project_access_view is to allow the user to view a project's data without direct access to the data tables themselves. The expected result of granting this permission is to allow the user to use SELECT queries on the project access view, but not modify the data or access other data. The command used for this would be:
 - GRANT SELECT ON research_group_db.project_access_view TO 'user@localhost'; FLUSH PRIVILEGES;
- The purpose of creating different roles such as project lead and researcher are to represent different types of users for the database. Roles allow for permissions to be shared with multiple users based on their role rather than adjusting one by one. This can be done with these two commands:
 - CREATE ROLE 'project_lead';
 - CREATE ROLE 'researcher';
- The purpose of granting privileges to roles is to allow different types of users to perform different commands and view different data. Project leads are given access to both views, while researchers are only allowed to view project data. This can be performed with the following commands:
 - GRANT SELECT ON research_group_db.member_contact_view TO 'project_lead'; GRANT SELECT ON research_group_db.project_access_view TO 'project_lead';
 - GRANT SELECT ON research_group_db.project_access_view TO 'researcher';

4. Identify and explain each non-conforming element in your database schema. Describe how it violates normalization principles and why it needs to be addressed.

One change that we noticed we needed to make was in the literature table. The authors column was defined as a single varchar(255) field but was being used to store multiple values. Specifically, several columns had multiple author names separated by commas (ex. "Jinek M, Chylinski K, Fonfara I, Hauer M, Doudna JA, Charpentier E"). This violates 1NF (first normal form) since the column is not atomic, but rather holds multiple values in a single field. 1NF requires that every column contain only one value per row, and each author is a separate piece of information. Because of this, querying for a specific author would require using LIKE '%author_name%' instead of a direct lookup. There is also no clean way to count how many papers an author contributed to, nor is there a way to link an author to their person

record if they're also a member of the department. Extracting authors into their own table resolves this violation and brings the database into compliance with 1NF.

5. Provide a detailed explanation of changes made during the normalization process. Justify each modification, explaining how it improves database efficiency, eliminates redundancy and inconsistencies, and ensures 3NF compliance.

To fix the issue with the `authors` column, we created two new tables, `author` and `literature_author`, and removed the `authors` column from the `literature` table.

```
25 --
26
27 --
28 -- Table structure for table `author`
29 --
30
31 CREATE TABLE `author` (
32   `author_id` int(5) NOT NULL,
33   `name` varchar(100) COLLATE utf8_unicode_ci DEFAULT NULL,
34   `person_id` int(5) DEFAULT NULL
35 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
36
37 --
38 -- Dumping data for table `author`
```

```
336 --
337
338 --
339 -- Table structure for table `literature_author`
340 --
341
342 CREATE TABLE `literature_author` (
343   `lit_id` int(5) NOT NULL,
344   `author_id` int(5) NOT NULL
345 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
346
347 --
348 -- Dumping data for table `literature_author`
349 --
350
351 INSERT INTO `literature_author` (`lit_id`, `author_id`) VALUES
```

Now each author has their own row in the `author` table, which fixes the 1NF violation since every column is atomic. The `literature_author` table connects authors to their literature entries. Since a paper can have many authors and an author can write many papers, this relationship is M:M and needs its own junction table, consistent with how we handle similar relationships throughout our schema. The `person_id` field on `author` is optional; if an author is also a member of a department, their author record can be linked directly to their `person` record; if they're external, then `person_id` is left as `NULL`. This eliminates redundancy, makes author lookups straightforward, and ensures 3NF compliance.

Submission

When you're finished, complete the following steps to submit your work:

- ☐ Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:
 - [Google Docs](#) (File → Download → PDF Document)
 - [Microsoft Word](#) (File → Save As / Export → PDF)
 - [Pages](#) (File → Export To → PDF)
- ☐ Inside your repo, export your updated database as an **SQL file** into the **db** folder. Follow the steps in the [Exporting A Database section](#) to export your database into a single SQL file.
- ☐ All group members should then be able to pull this SQL file from the repo and upload it to their own cPanel accounts (see [Importing A Database section](#)).
- ☐ Upload all your changes to GitHub.
 - ☐ **If you're using GitHub Desktop (GUI)**, complete the [Uploading Changes \(GitHub Desktop\) section](#) to upload your changes from your local device to GitHub.
 - ☐ **If you're using Git (CLI)**, complete the [Uploading Changes \(GitHub CLI\) section](#) to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Database Views & Normalization grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.